

Lesson 7 Sidebars

On the Move

VIDEO GAME CREATION

These aren't part of the lesson's required work. Think of them the way a textbook sidebar works: a deeper, tangential look at something this lesson's build actually ran into - why one approach was chosen over the others, what the alternatives were, or an idea worth a longer look than the Handout has room for. Pull one out for a discussion moment if it fits, hand it to a curious student, or skip it entirely - nothing here is required reading.

A Destroy Event Doesn't Know Why It Fired

This lesson quietly fixed a real bug that had been sitting in two engines since Lesson 6 - and it's a genuinely common, real-world mistake, not a beginner one.

Lesson 6 needed one thing: decrement the win-counter when an enemy is defeated. In most engines, that logic naturally landed wherever the enemy's removal was already being handled - a signal, a direct function call, a collision event specifically tied to being hit.

In GameMaker and 001 Game Creator specifically, the natural-looking place for that logic was a generic lifecycle event - GameMaker's Destroy event, 001 Game Creator's "Defeated / Destroyed" trigger. Both fire for ANY reason an instance gets removed, not specifically for being hit. That worked perfectly in Lesson 6, because being hit was the only way an enemy could ever be destroyed. The instant Lesson 7 added a second way to remove an enemy - escaping off-screen unshot - the generic event started firing for both cases, and an escaped enemy would silently still count toward the win condition. A real bug: nothing crashes, nothing looks obviously wrong, the game just becomes winnable in a way it shouldn't be.

The fix, and why it generalizes past these two engines: move the win-counting logic specifically onto the ONE event that actually, specifically represents a hit - not a generic "this object is gone now" lifecycle event. Stated as a general rule: a generic lifecycle event answers "did this happen," not "why did this happen." If the why matters - and for anything tied to scoring, counting, or game state, it almost always does - that logic belongs on the specific event that captures the reason, never the generic one.

Worth naming why the other six engines never had this bug at all: Godot's signal, Unity's direct call, GDevelop's collision event, TIC-80's separate loop, and Unreal's dispatcher were all already scoped specifically to "this was a hit" from the moment Lesson 6 was written - there was never a generic "any destruction" event sitting there, tempting enough to reach for by accident. That's a real, structural difference in what a given engine makes easy by default versus easy to get subtly wrong.

That Yellow Warning in the Debugger: What UNUSED_PARAMETER Actually Means

A warning isn't an error - the game runs exactly the same either way. Worth knowing the difference anyway, since Godot prints all three very different kinds of complaint through the same-looking little panel.

Leave one of Godot's default stubbed functions in a script - _process(delta) is the classic case, auto-filled in by the editor the moment a script is created - and if the body never actually reads delta, Godot's own checker prints exactly this in the Debugger the next time the project runs: "The parameter 'delta' is never used in the function '_process()'. If this is intended, prefix it with an underscore: '_delta'." Nothing crashes, nothing behaves differently, and the game runs identically whether that warning is fixed or ignored forever - it's a heads-up, not a problem.

The recommended fix is exactly what the warning says: rename the parameter from delta to _delta, everywhere it appears in that function's signature. That's a pure rename - _delta holds the exact same value, at the exact same moment, as delta did; nothing about how the function actually runs changes at all. What does change is that GDScript's own checker specifically watches for a leading underscore on a parameter name, and treats it as a deliberate signal: "yes, I know this parameter exists and isn't used - that's on purpose, stop flagging it." The underscore isn't doing anything to the code itself. It's a note left for the tool that's reading the code, not for the computer that's running it.

Three Different Kinds of Rule, Worth Telling Apart

That distinction - a note for a tool, not a rule the computer enforces - is worth pulling apart properly, because Godot's Debugger genuinely mixes three very different categories of complaint into what looks like one kind of message:

- **A coding convention:** a habit programmers agree to follow because it helps people read the code later - this course's own Standards Rule 11 (meaningful variable names over generic i/j/k) is exactly this. Nothing about the language checks it, nothing enforces it, and breaking it doesn't stop the code from running - it just makes the code harder for a person to follow.
- **A compiler/language-enforced rule:** a rule GDScript itself requires. Break one - a missing colon, a variable used before it's declared, mismatched indentation - and the script genuinely will not run at all. This is the only category where the computer refuses to proceed.
- **A static analyzer / linter warning:** the middle ground, and where UNUSED_PARAMETER actually lives. The code is completely valid GDScript and will run exactly as written - but a separate tool built into Godot's editor scans it for patterns that are usually mistakes (an unused parameter often means a forgotten piece of logic) and flags them as a courtesy, without stopping anything. The underscore prefix is GDScript's own built-in way of telling that specific tool, "I checked, this one's intentional."

Worth flagging one more thing, without getting into the full explanation yet: this exact same leading-underscore character shows up elsewhere in GDScript too, meaning something related but not identical - a variable or method name starting with an underscore is a separate convention, hinting that it's meant to stay internal to that one script rather than be reached from

outside it. Different meaning each time the underscore shows up, same basic idea underneath both: a leading underscore is how GDScript-flavored code signals "this is intentional, on purpose, look here for why" to whoever - or whatever tool - reads it next.

Renaming the parameter rather than deleting the function entirely is exactly the right instinct here, too - an unused _process(_delta) costs nothing to leave in place, and it's already sitting there, ready the moment there's real per-frame logic to add.